

A Federated Hong Kong Transport Dataspace

Real-World Datasets, Verifiable-Credential-Driven Policy,
and Live Issuance on Eclipse Dataspace Components

Li Chudikang Zhao Yuxuan
Student ID: 122040057 Student ID: 124090921

May 2026

Course: CSC4240 Data Spaces (and CSC6203 Selected Topics in CS (III) Data Spaces)
Instructor: Prof. George Polyzos
Institution: The Chinese University of Hong Kong, Shenzhen
Source code: <https://github.com/LiChudikang/csc4240-hk-federated-dataspace>

Abstract

Hong Kong transport data is rich but siloed: KMB, MTR, the Transport Department, and the Observatory each publish open or paid feeds, but each lives behind a separate contract, rate limit, and access discipline. Bilateral integration does not scale, and as policy questions about privacy, attribution, and downstream redistribution sharpen, the gap is increasingly visible.

We design and implement a federated Hong Kong transport dataspace on the Eclipse Dataspace Components (EDC) Minimum Viable Dataspace, with three semantic participants (one in a dual role), a Federated Catalog Node aggregating four publishing connectors into a single 12-dataset catalog, real upstream data from `data.etabus.gov.hk`, and two custom ODRL policy functions that key access off issuer-attested verifiable-credential claims rather than off holder identifiers. We further demonstrate live DCP credential issuance and S3-compatible push transfer end-to-end. All claims are backed by seven demonstrations realised through eight reproducible scripts and twenty-four captured evidence files.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Project goals	3
1.3	Related dataspace and applications	3
1.4	Contributions	3
1.5	Report outline	4
2	Background	4
2.1	The dataspace abstraction	4
2.2	Eclipse Dataspace Components	4
2.3	Protocols: DSP, DCP, ODRL	5
2.4	Trust model: DIDs and Verifiable Credentials	5
3	Scenario and Design	5
3.1	The Hong Kong transport scenario	5
3.2	Actors	5
3.3	Data assets	5

3.4	Credentials in this deployment	6
3.5	Federated topology	6
4	System Architecture	7
4.1	Deployment topology	7
4.2	End-to-end flow	7
5	Implementation	7
5.1	Stock MVD baseline (Part 1 of the assignment)	7
5.2	Custom EDC extensions	8
5.3	Verifiable Credential lifecycle	9
5.4	Federated catalog crawler	10
5.5	Push transfer to S3 (MinIO)	10
6	Data	10
6.1	Sources	10
6.2	Asset inventory	10
7	Evaluation	10
7.1	Demo 1 — Negotiation and HTTP-PULL transfer	11
7.2	Demo 2 — Push transfer to S3 (MinIO)	11
7.3	Demo 3 — HKU dual role	11
7.4	Demo 4 — Federated catalog crawl	12
7.5	Demo 5 — Live DCP credential issuance	12
7.6	Demo 6 — Custom Permission constraint (<code>participantTier</code>)	12
7.7	Demo 7 — Attribution Duty	12
8	Discussion	12
8.1	Problems overcome	12
8.2	Limitations	13
8.3	Lessons learned	13
9	Conclusion and Future Work	14
9.1	Conclusion	14
9.2	Future work	14
10	Code Repository	14
A	Demo script index	15

1 Introduction

1.1 Motivation

Hong Kong’s mobility ecosystem is a textbook case of *rich data, missing plumbing*. The Transport Department publishes open data on traffic volume, speed, and incidents; KMB exposes a public real-time arrival API at `data.etabus.gov.hk`; MTR releases patronage and station-throughput statistics; the Observatory contributes weather and air-quality streams that modulate everything above. Each feed is technically reachable, but their *combination* — the operation a route-planning app, a transit-equity study, or a pandemic-mobility dashboard actually needs — requires bilateral licensing, ad-hoc credential management, custom rate-limit handling, and manual attribution

discipline. There is no shared catalog, no shared identity layer, and no machine-enforceable usage policy beyond “you signed a PDF”.

As traffic data grows in volume and policy questions sharpen — privacy of movement, attribution of derived analytics, downstream redistribution, the liability of consumers who re-package public-feed data into commercial products — this bilateral model does not scale. A *dataspace* [1] is the architectural answer the IDSA / Gaia-X / Eclipse Dataspace Components communities have converged on: keep data sovereign at the producer, but standardise the discovery, contracting, transfer, and enforcement layers so that joining the federation means *speaking the protocol*, not signing N separate contracts.

1.2 Project goals

This project realises a small but complete federated dataspace for Hong Kong transport data. Concretely it must (a) instantiate the IDSA reference flow end-to-end with real upstream data, (b) include at least one dual-role participant, (c) expose a Federated Catalog Node that crawls every provider on a configurable interval, (d) enforce policies that key off *verifiable-credential claims* — not on participant identifiers — and (e) demonstrate live credential issuance over DCP. Performance benchmarking and production hardening (TLS, KMS-backed keys, high-availability storage) are explicit non-goals; we discuss them in §9.

1.3 Related dataspace and applications

We position our work against four reference points.

IDSA Reference Architecture Model 4.0 [1] is the canonical specification we conform to: it defines the role separation (connectors, identity providers, clearing house, vocabulary providers), the DSP message catalog, and the policy-language attachment via ODRL.

Eclipse Dataspace Components (EDC) [2] is the most actively-developed open-source implementation of the IDSA RAM in production-grade Java; the Minimum Viable Dataspace (MVD) [3] is its canonical example deployment, shipping a Catalog Server, identity hubs, and a standalone issuer service. Our work is a derivative of MVD.

Mobility Data Space (MDS), Germany [4], operated by DRM Datenraum Mobilität GmbH, is the closest production analogue to our scenario: a federated mobility dataspace whose participants negotiate over IDS-style contracts. MDS, however, currently relies on bilateral contract templates rather than fine-grained, machine-evaluated ODRL constraints, and does not publicly expose its policy functions. Our project demonstrates that the finer-grained model is feasible at the size of a course project.

Catena-X [5] is the largest currently-deployed dataspace on the EDC stack, in the automotive supply-chain domain. It demonstrates verifiable-credential-driven access at industrial scale; we adopt the same trust pattern (issuer-attested membership credentials feeding into the PolicyEngine) at small scale, on transport data instead.

To our knowledge no academic prototype combines IDSA-compliant policy enforcement with real Hong Kong transport upstreams; existing transport-data sharing studies either skip the policy layer or stop at API mediation.

1.4 Contributions

This report describes the following concrete contributions:

1. A federated topology with three semantic actors — *HK Transport Hub* (the provider, internally split across two connectors plus a federated catalog server), *HKTaxi* (a commercial consumer), and *HKU Transport Lab* (a dual-role academic participant) — spread across 21 pods in a K3d cluster.
2. A Federated Catalog Node that periodically crawls every provider connector and exposes a unified catalog containing 12 datasets across 4 providers (§6).

3. Real-world data integration: live KMB ETA JSON pulled on demand from the public `data.etabus.gov.hk` endpoint, KMB route master, plus realistic HK Transport Department, MTR-patronage, and HKU research-archive assets registered as catalog offers.
4. **VC-claim-driven access control.** A custom ODRL constraint (`participantTier`) backed by an EDC `AtomicConstraintRuleFunction` that reads tier from a *MembershipCredential* claim issued by the dataspace issuer, rather than substring-matching on the holder DID.
5. An ODRL **Duty** (`AttributionDutyFunction`) that the consumer must satisfy on every pull, demonstrating obligation-style policies.
6. Live DCP credential issuance: HKU obtains a fresh `FoobarCredential` on demand from the issuer service, exercising the `CREATED→REQUESTING→REQUESTED→ISSUED` state machine end-to-end.
7. Push transfer to S3-compatible storage (MinIO deployed in-cluster), exercising the EDC `AmazonS3-PUSH` transfer type.
8. Reproducible deployment: K3d + Terraform + Helm, plus seven demonstrations realised through eight scripts that exercise every claim above, with evidence files captured under `final/screenshots/` for after-the-fact inspection.

1.5 Report outline

§2 surveys background concepts. §3 presents the design of the dataspace. §4 describes the system architecture and deployment topology. §5 walks through the implementation, focusing on the custom EDC extensions written for this project. §6 documents the data sources and asset inventory. §7 evaluates the system through seven end-to-end demonstrations. §8 discusses problems overcome and limitations. §9 concludes and lists future work. §10 points to the public code repository.

2 Background

2.1 The dataspace abstraction

A *dataspace* [1] is a federation of independent participants who share data under machine-enforceable usage policies. Three properties distinguish it from a data platform or a federated query engine:

- **Sovereignty.** Data stays at the producer; access is brokered via a contract, not copied to a central store.
- **Decentralised identity.** Participants prove who they are via Decentralised Identifiers (DIDs) [7] and Verifiable Credentials issued by trusted authorities, not via central IAM.
- **Policy enforcement at the connector.** Usage policies travel with the data (in the contract agreement) and are checked by both sides' connectors before, during, and after transfer.

2.2 Eclipse Dataspace Components

EDC [2] is a modular Java framework. The pieces relevant to this project are:

- **Connector** (Control Plane + Data Plane) — speaks DSP, negotiates contracts, orchestrates transfers.
- **Catalog Server** — a connector specialised to act as a federated catalog node, periodically crawling other connectors' offers.

- **Identity Hub** — stores Verifiable Credentials for a participant; speaks DCP to credential issuers and to verifiers.
- **Issuer Service** — a standalone runtime that issues VCs using the DCP state machine.

2.3 Protocols: DSP, DCP, ODRL

Three protocols carry all interactions in our deployment:

- **Dataspace Protocol (DSP)** — HTTP-based; defines the `CatalogRequest`, `ContractRequest`, and `TransferRequest` message exchanges and their state machines.
- **Decentralised Claims Protocol (DCP)** — used both for credential *issuance* (which we demonstrate live) and for credential *presentation* during contract negotiation.
- **Open Digital Rights Language (ODRL)** [6] — the JSON-LD vocabulary used to express policies as a triple of permission, prohibition, and duty, each with optional constraints.

2.4 Trust model: DIDs and Verifiable Credentials

Every participant has a `did:web` identifier resolvable, in our cluster, to a DID document hosting the participant’s public key. The dataspace issuer (`did:web:dataspace-issuer-service%3A10016:issuer`) issues `MembershipCredentials`, modelled per the W3C Verifiable Credentials data model [8], naming the holder and a participant tier (`COMMERCIAL`, `ACADEMIC`, or `PROVIDER`). These credentials are presented as JWT verifiable presentations during DSP negotiation and are the input to our policy functions (§5.2).

3 Scenario and Design

3.1 The Hong Kong transport scenario

Three semantic actors share data through the dataspace.

HK Transport Hub is the data provider. In a production deployment it would be a Hong Kong governmental or industry organisation (for example, the Transport Department or a designated data trustee) that aggregates and re-publishes operator feeds. In our deployment the Hub is split across three connectors: `provider-qna` (KMB live ETA and route master), `provider-manufacturing` (HK Traffic Department incidents, MTR patronage, academic-research archive), and `provider-catalog-server` (the federated catalog node that crawls the other two and is itself a catalog publisher).

HKTaxi is a commercial consumer — a route-optimisation app that needs the KMB feed to recommend trips. It holds a `MembershipCredential` with `participantTier=COMMERCIAL`.

HKU Transport Lab is a dual-role academic participant. As a consumer, it pulls KMB and traffic data for transit-equity research; as a provider, it publishes a research-archive asset back into the federation that other participants can negotiate against. Its `MembershipCredential` carries `participantTier=ACADEMIC`.

A *Dataspace Issuer Service* sits outside the producer/consumer split and acts as the trust anchor: it issues all three actors their `MembershipCredential` and, on demand, additional credential types (§7.5). The relationships are summarised in Figure 1.

3.2 Actors

3.3 Data assets

Twelve datasets are visible in the federated catalog. The full inventory is listed in §6; design-wise, the assets are partitioned into:

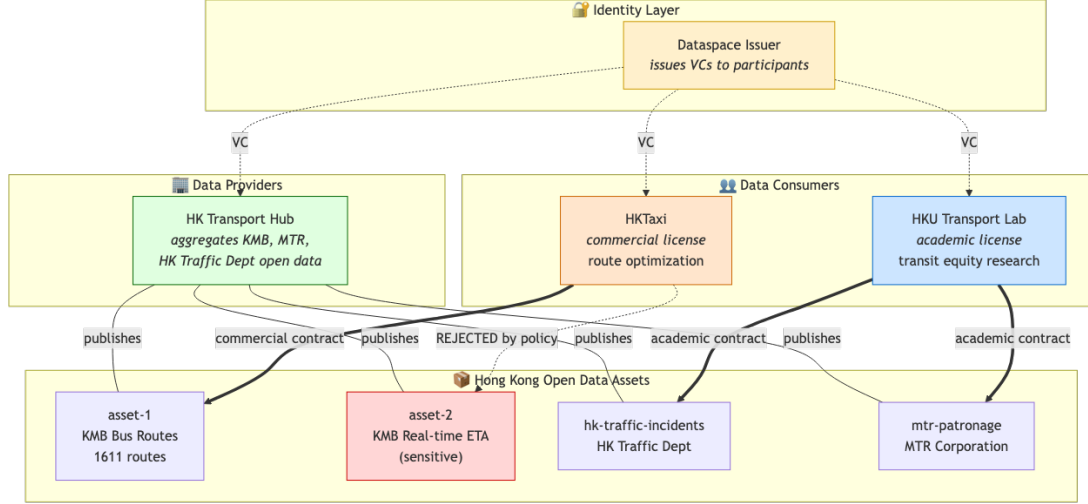


Figure 1: Scenario topology. *HK Transport Hub* (green) publishes four HK transport datasets. *HKTaxi* (orange) has a commercial contract for the KMB routes asset but is rejected by policy when it requests the sensitive ETA asset. *HKU Transport Lab* (blue) holds an academic contract for the traffic-incidents and MTR-patronage assets. The *Dataspace Issuer* (yellow) issues a Verifiable Credential to each participant, which is presented during contract negotiation.

- *Open transport data* from KMB and the HK Transport Department, available to consumers carrying any tier.
- *Sensitive transport data* (real-time ETA, MTR patronage), gated by the `participantTier` permission constraint.
- *Academic-research datasets*, tagged for academic consumption only; the HKU dual-role asset (`hku-transit-equity-2026`) belongs here.
- *Attribution-bearing data*, which carries an additional ODRL duty (`asset-with-attribution`; see §7.7).

3.4 Credentials in this deployment

Building on the generic DID/VC trust model summarised in §2.4, three credential types drive policy decisions in our scenario:

- `DataProcessorCredential` (preloaded) — gates whether a holder may participate in the dataspace at all.
- `MembershipCredential` (preloaded; carries the `participantTier` claim) — drives the custom permission constraint.
- `FoobarCredential`¹ (issued live during the demo) — exercises the DCP issuance flow.

The credentials are issued at deploy time by the Dataspace Issuer Service into each participant’s identity hub via a Helm configmap, except for `FoobarCredential`, which is requested by HKU at runtime.

3.5 Federated topology

A single Catalog Server crawls `provider-qna`, `provider-manufacturing`, and the HKU provider-side connector at a fixed interval (configured via `edc.catalog.cache.execution.period.seconds`), indexes their offers, and exposes a unified catalog endpoint. A consumer issues a single catalog

¹We adopt the upstream MVD’s `FoobarCredential` type for live issuance to keep the issuer code unmodified; in production this would be a domain-specific type such as `AcademicAffiliationCredential`.

Table 1: Actors and their connectors. The semantic role names (HK Transport Hub, HKTaxi, HKU) drive the narrative; the connector names (`provider-*`, `consumer`, `hku`) are the technical identifiers used in the EDC configuration and in the demo scripts.

Semantic actor	Connector(s)	Role / DID (<code>did:web</code> resolved by cluster DNS)
HK Transport Hub	<code>provider-qna</code>	Publishes KMB ETA / route master. <code>did:web:provider-identityhub%3A7083:provider</code>
HK Transport Hub	<code>provider-manufacturing</code>	Publishes HK traffic incidents, MTR patronage, academic-research archive. (shares the <code>provider</code> identity hub)
HK Transport Hub	<code>provider-catalog-server</code>	Federated Catalog Node; periodically crawls the other connectors.
HKTaxi	<code>consumer</code>	Commercial consumer. <code>did:web:consumer-identityhub%3A7083:consumer</code>
HKU Transport Lab	<code>hku</code>	Academic consumer + provider (dual role). <code>did:web:hku-identityhub%3A7083:hku</code>
Dataspace Issuer	<code>dataspace-issuer-service</code>	Trust anchor; issues MembershipCredentials. <code>did:web:dataspace-issuer-service%3A10016:issuer</code>

request and receives the union — in our deployment, 12 offers — without knowing the underlying provider topology.

4 System Architecture

4.1 Deployment topology

The dataspace runs on a local single-node K3d cluster, provisioned via Terraform + Helm. The `mvd` namespace contains 21 pods, organised in the four sub-clusters shown in Figure 2: the provider side (catalog server, two provider connectors, identity hub, vault, Postgres), two consumer sides (HKTaxi and HKU, each with control plane, data plane, identity hub, vault, Postgres), and the identity sub-cluster (issuer service, DID server, issuer Postgres).

Cluster-internal DNS (`hku-identityhub`, `dataspace-issuer-service`, `provider-identityhub`, ...) resolves the `did:web` identifiers; `ingress-nginx` exposes selected endpoints at `http://127.0.0.1/<actor>/<role>/...` so that the demo scripts can drive negotiation, transfer, and credential issuance from a developer laptop.

4.2 End-to-end flow

Figure 3 shows the seven-step flow that Demo 1 exercises. The same skeleton, with different transfer types and different VPs presented, underpins all seven evaluations in §7.

5 Implementation

We deliberately *describe* code in this section rather than reproduce it; the public repository (§10) contains the full sources.

5.1 Stock MVD baseline (Part 1 of the assignment)

Before extending the MVD we ran the canonical 2-connector setup (`provider-qna`, `consumer`) end-to-end: catalog discovery → contract negotiation → HTTP-PULL transfer, plus a separate HTTP-PUSH transfer. This validated our local toolchain (K3d, Helm, Postman / curl, jq) and

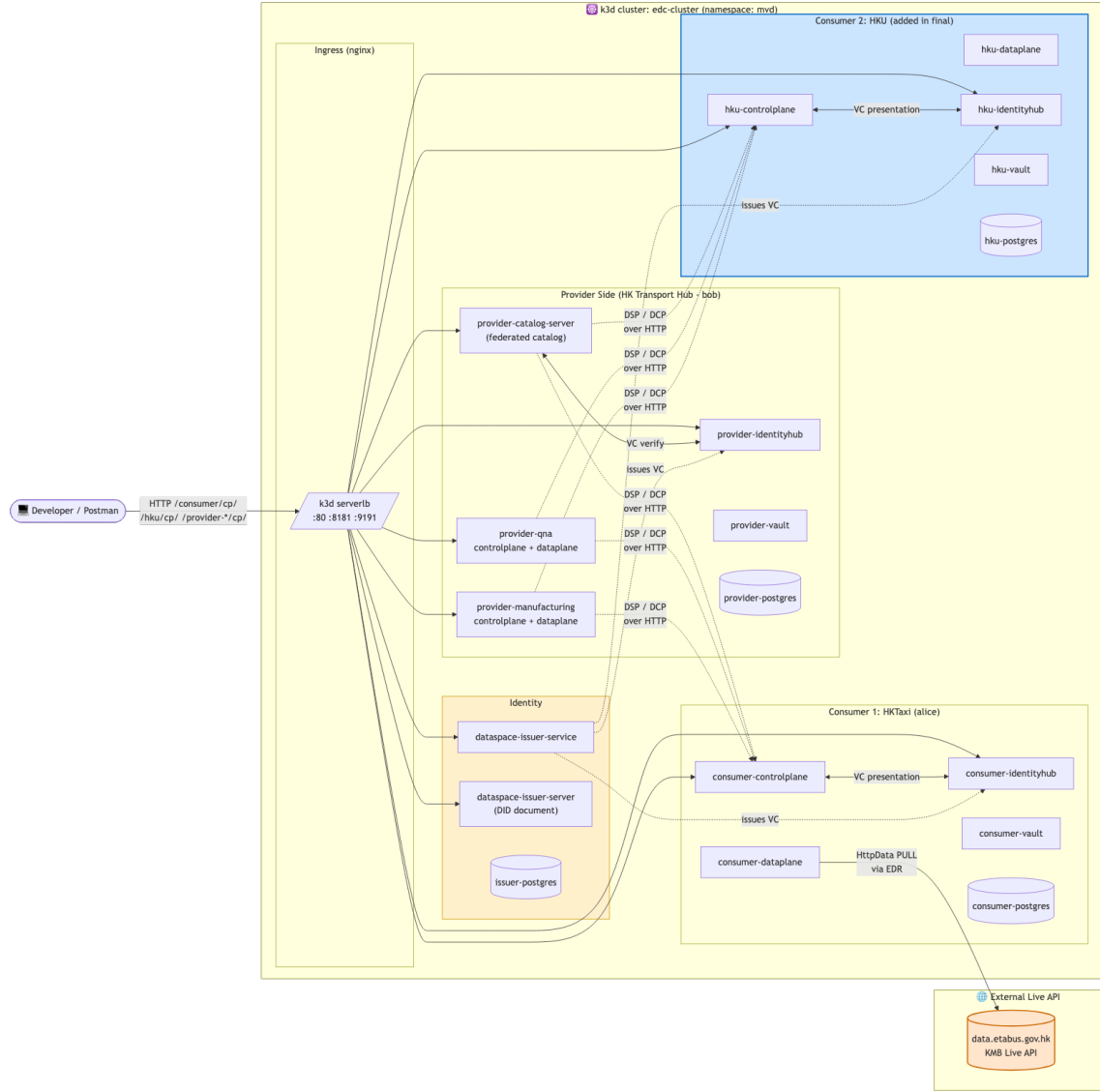


Figure 2: K3d deployment architecture. Each participant has its own control plane, data plane, identity hub, vault, and Postgres. The Dataspace Issuer Service issues VCs into all three identity hubs. The HKU sub-cluster (highlighted in blue) was added in the final iteration as the third participant. Outside the cluster, the consumer dataplane fetches live KMB JSON via the EDR-authorized pull path (Demo 1).

gave us reference logs to compare our extensions against.

5.2 Custom EDC extensions

Our policy contributions live under `extensions/dcp-impl` as **two new AtomicConstraintRuleFunction implementations plus the registration glue** that hooks them into EDC's PolicyEngine:

- `ParticipantTierFunction` is an `AtomicConstraintRuleFunction<Permission>` bound to the ODRL property `participantTier`. On each policy evaluation it receives the `ParticipantAgent` (built from the presented VP), looks up `MembershipCredential` VCs in the agent's claims, extracts the `participantTier` field, and matches it against the right operand. This replaces an earlier DID-substring approach we discarded as unsafe (see §8 problem 1).
- `AttributionDutyFunction` is an `AtomicConstraintRuleFunction<Duty>`. Bound to a duty

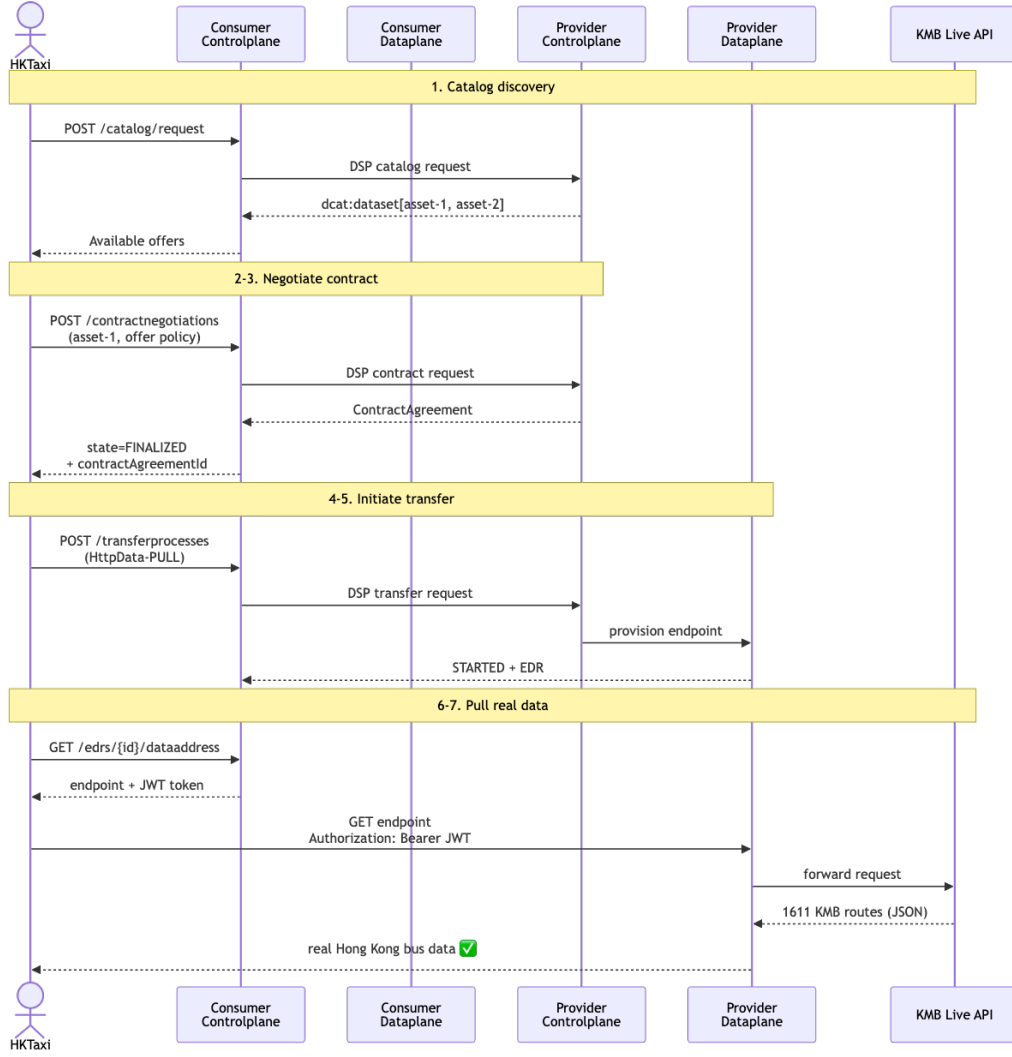


Figure 3: End-to-end pipeline of Demo 1: catalog discovery, contract negotiation, transfer initiation, EDR-token retrieval, and finally data fetch. Steps 6–7 produce the live KMB JSON. Push and live-issuance variants substitute steps 4–5 with different message bodies.

whose left operand is `attribution`, the function requires the consumer to declare an attribution string before each pull and verifies that the required marker is present in the duty payload. Demonstrates obligation-style ODRL: the policy is satisfied not by a property of the holder but by an action the holder commits to.

- `PolicyEvaluationExtension` (modified) — the existing `dcp-impl` registration extension was extended to register the two functions above with the appropriate policy scopes (negotiation, transfer, catalog).

For DCP issuance we did *not* write a new attestation source: EDC ships a demo attestation factory (`DemoAttestationSource` / `DemoAttestationsExtension`, upstream Cofinity-X code) that we leverage as-is. Our contribution there is a data-side fix — inserting a row in the issuer’s `membership_attestations` Postgres table for HKU so that the attestation pipeline does not NPE when HKU requests a credential (§8 problem 4).

5.3 Verifiable Credential lifecycle

Two paths coexist in the deployment:

- **Preloaded VCs** (used for routine demos): each participant’s identity hub is seeded

at deploy time from a Helm configmap containing `DataProcessorCredential` and `MembershipCredential` JWTs signed by the issuer.

- **Live issuance** (Demo 5): HKU sends a `CredentialRequestMessage` via DCP; the issuer service evaluates an attestation pipeline (Postgres lookup + the demo source), signs a fresh `FoobarCredential` JWT, and pushes it back to HKU's identity hub.

The preloaded `MembershipCredential` JWTs were re-signed with the issuer's Ed25519 private key to inject the `participantTier` claim; the resulting tokens are checked into `deployment/assets/credentials/k8s/<role>/membership-credential.json`.

5.4 Federated catalog crawler

The Catalog Server is configured (via `edc.catalog.cache.execution.period.seconds`) to crawl each provider connector at a fixed interval. Targeted DSP endpoints are listed in a configmap (one entry per provider); the server materialises a unified catalog and exposes it via the same DSP catalog endpoint that any single-provider connector exposes.

5.5 Push transfer to S3 (MinIO)

We deployed a MinIO stateful service in-cluster with a single bucket (`edc-push-bucket`) and added the `data-plane-aws-s3` EDC extension (version 0.7.0; see §8 problem 5) to the dataplane build. Demo 2's transfer uses `transferType: "AmazonS3-PUSH"` and a `dataDestination` carrying `endpointOverride`, `accessKeyId`, and `secretAccessKey`. The provider's dataplane fetches the upstream KMB JSON and writes it to the bucket.

6 Data

6.1 Sources

The dataspace mixes three categories of data: live external feeds, snapshot files of public Hong Kong transport data, and synthetic but realistic research archives.

- **Live KMB ETA / routes**: pulled at request time from the public KMB open-data endpoint at `data.etabus.gov.hk` [9]. Latencies are typically below 200 ms; a single ETA request returns roughly 350 KB of JSON, a single route request roughly 250 KB.
- **HK Transport Department incidents** and **MTR patronage**: stored as static JSON files under the provider connector's asset registry. The schemas mirror the published HKTD and MTR open-data formats; the snapshots are dated to allow reproducibility.
- **HKU research archive**: a synthetic but realistic CSV representing a transit-equity study, registered as the HKU dual-role asset (`hku-transit-equity-2026`).

6.2 Asset inventory

The federated catalog crawler observed the twelve datasets shown in Table 2 during the run captured in `screenshots/14-federated-crawler.txt`. Multiple connectors share the `asset-1` / `asset-2` naming convention because each was seeded with the canonical demo identifiers; the federation index disambiguates them by provider name.

7 Evaluation

We exercise the system through seven scripted demonstrations, implemented as eight scripts under `final/scripts/` (Demo 3 uses two). Each demonstration captures evidence files (JSON /

Table 2: Federated catalog inventory. All entries are `dcat:Dataset` offers exposed via the DSP catalog endpoint.

#	Provider	Asset ID	Description
1	qna	asset-1	KMB live route master (1,611 routes)
2	qna	asset-2	KMB real-time ETA (sensitive; tier-gated)
3	qna	asset-1-push	KMB route master, push variant
4	qna	asset-tiny-push	Small JSON sample for push smoke test
5	qna	asset-with-attribution	Carries an attribution duty (Demo 7)
6	manufacturing	asset-1	HK Transport Department open dataset
7	manufacturing	asset-2	HK Transport Department incidents
8	manufacturing	hk-traffic-incidents	Real-time traffic incident snapshots
9	manufacturing	mtr-patronage	MTR daily patronage statistics
10	manufacturing	hk-academic-research-archive	Cross-cited research dataset
11	catalog-server	normal-asset-1	Catalog-server-resident demo asset
12	hku	hku-transit-equity-2026	HKU dual-role research archive

plain-text log) under `final/screenshots`² so results are reproducible after the fact. The scripts are listed in Appendix A.

Evaluation here is functional rather than performance-oriented; quantitative benchmarking is an explicit non-goal of this project, and where we cite numbers (payload sizes, observed latencies) they are single illustrative measurements, not statistical claims.

7.1 Demo 1 — Negotiation and HTTP-PULL transfer

`demo-12-transfer.sh`. HKTaxi’s connector fetches the catalog from the Catalog Server, selects the KMB live-ETA offer, negotiates a contract, requests a transfer in pull mode, exchanges an EDR, and finally pulls live KMB JSON from the provider’s data plane. The full seven-step flow is shown in Figure 3.

Evidence: 01-catalog.json, 02-negotiation-finalized.json, 03-transfer-started.json, 04-edr.json, 05-real-kmb-data.json.

7.2 Demo 2 — Push transfer to S3 (MinIO)

`demo-push-s3.sh`. The same negotiation as Demo 1 but with `transferType: AmazonS3-PUSH`. The provider’s dataplane writes a 352 KB KMB JSON object into the in-cluster MinIO bucket; the script verifies the object via `mc ls` inside the MinIO pod.

Evidence: 22-s3-push-bucket-listing.json, 23-s3-push-evidence.txt.

7.3 Demo 3 — HKU dual role

`demo-hku-dual-role.sh` + `demo-hku-as-consumer.sh`. HKU first publishes its research asset and exposes it via DSP; HKTaxi then negotiates with HKU, gets an EDR, and pulls the HKU data; finally HKU itself acts as a consumer and pulls KMB data from `provider-qna`. This proves both directions of the dual-role pattern in a single run.

Evidence: 13a-hku-as-provider-catalog.json through 13d-hku-research-data.json, 16-hku-consumer-catalog.json, 17-hku-academic-agreement.json, 18-hku-academic-data-pulled.json.

²The directory is named `screenshots/` for historical reasons; in practice it contains JSON responses and plain-text command logs rather than screen captures.

7.4 Demo 4 — Federated catalog crawl

`federated-crawler.sh`. The Catalog Server’s unified catalog is queried once; the response lists offers from `qna`, `manufacturing`, `catalog-server`, and `hku`, demonstrating that the crawler ran on its configured cadence and merged the four underlying catalogs (Table 2).

Evidence: `14-federated-crawler.txt`.

7.5 Demo 5 — Live DCP credential issuance

`demo-live-issuance.sh`. HKU’s identity hub initiates a `CredentialRequestMessage` to the issuer service. The DCP state machine progresses `CREATED`→`REQUESTING`→`REQUESTED`→`ISSUED`, verified by tailing the identity hub log filtered on the request ID. The issuer evaluates an attestation pipeline (Postgres membership lookup + the demo attestation factory) and signs a fresh `FoobarCredential` JWT, which is then pushed into HKU’s wallet. The script decodes the resulting JWT and confirms `iss` = issuer DID, `sub` = HKU DID, `iat` = the demo’s timestamp (i.e. not the configmap-baked 2023-12-12 of preloaded VCs), and `vc.type` including `FoobarCredential`.

Evidence: `24-live-credential-issuance.txt`.

7.6 Demo 6 — Custom Permission constraint (`participantTier`)

`demo-custom-constraint.sh`. Two flows:

- HKTaxi (`participantTier=COMMERCIAL`) succeeds in pulling a `commercial-only` asset.
- HKU (`participantTier=ACADEMIC`) is rejected with an explicit policy-violation message when requesting the same asset.

This proves the rule reads the VC claim, not the holder DID: if it were reading the DID, both holders would be indistinguishable to the rule.

Evidence: `08-policy-rejection.json`, `15-custom-constraint-rejection.json`, `17-hku-academic-agreement.json` (positive case for the `ACADEMIC` asset).

7.7 Demo 7 — Attribution Duty

`demo-attribution-duty.sh`. The asset’s policy carries a duty: “on use, declare an attribution string”. When the consumer satisfies the duty (the attribution payload is present and well-formed), the pull succeeds; when it does not, the pull is rejected by `AttributionDutyFunction` before reaching the data plane.

Evidence: `19-attribution-duty-offer.json`, `20-attribution-success.json`, `21-attribution-rejection.json`.

8 Discussion

8.1 Problems overcome

1. **Tier-from-DID was not real policy enforcement.** Our first version of the tier check matched on substrings of the holder DID. This worked, but was a lie: the provider was trusting the *name* of the participant, not a credential issued about it. We replaced it with a VC-claim lookup (§5.2, `ParticipantTierFunction`), which required us to inject the tier claim into the preloaded `MembershipCredentials` and re-sign each one with the issuer’s `Ed25519` private key.
2. **DCP credential request body schema.** First attempts using `credentialType` as the credential descriptor returned HTTP 400. Reading the EDC end-to-end test (`tests/end2end/.../CredentialIssuanceEndToEndTest.java`) revealed that the correct shape uses `type` plus `id` referencing a credential definition.

3. **Issuer participant context stuck in CREATED.** The seeded issuer participant context started inactive; credential requests failed with HTTP 500 until activated via `POST /participants/{ctx}/state?isActive=true`. This is not obvious from the management-API documentation but is required once per fresh deployment.
4. **AttestationPipelineImpl.evaluate NPE for HKU.** The default seed only registered the consumer in `membership_attestations`; the attestation source returned null for HKU, NPEing inside the pipeline. We inserted an HKU row with `membership_type=3` as part of the cluster bootstrap.
5. **EDC AWS S3 module versioning.** `data-plane-aws-s3:0.14.1` does not exist on Maven Central: EDC's Technology-AWS modules are versioned independently of EDC core. We pinned to 0.7.0 and added a separate `edc-aws` version line in `libs.versions.toml`. The failure mode was a Gradle dependency-resolution error during dataplane build, rather than a runtime bug.
6. **participant_context_id is a legacy DID.** The issuer's Postgres still keys some tables on `did:web:localhost%3A10100` from a pre-K8s seed, not on the current `did:web:dataspace-issuer-service%3A10016:issuer`. Demos that touch the issuer admin API have to base64 the legacy DID for those particular path parameters.
7. **Custom ODRL constraints cannot be pure JSON.** Out of the box, EDC ships only a couple of built-in constraint functions; a property like `participantTier` requires a Java extension that registers the function with `PolicyEngine.registerFunction(...)`. This is documented in EDC, but worth flagging as a real limitation for adopters who hoped policy was config-only.

8.2 Limitations

- All endpoints are plain HTTP; no TLS termination, no mTLS, no certificate-based DID verification.
- Single-region, single-node K3d. No HA, no Postgres backups.
- Keys live in plain configmaps and Vault dev mode (in-memory, cleared on pod restart).
- Only one “real” upstream is wired (KMB); MTR / HKO weather were planned but stayed at the design stage.
- The Duty (§7.7 attribution) is checked synchronously by the consumer connector; production deployments would want an audit log on a separate trust domain so that the consumer cannot trivially fabricate satisfaction.

8.3 Lessons learned

- **EDC's E2E tests are the most reliable spec.** When documentation and code disagreed, the tests under `tests/end2end/...` were always right. This was particularly true for DCP message bodies (problem 2 above).
- **Credential plumbing is the time sink.** The policy *logic* took hours; getting the right VC into the right hub at the right time, and re-signed when claims changed, took days. Tooling for VC re-signing and verification would meaningfully reduce the cost of running a small dataspace.
- **did:web works locally if your DNS does.** Cluster-internal DNS plus consistent `did:web:<service>%3A<port>:<id>` naming let us drop `did:key` entirely and made every DID resolvable from any other pod in the cluster.

9 Conclusion and Future Work

9.1 Conclusion

We have built and demonstrated a federated Hong Kong transport dataspace that exercises every capability the assignment lists for Part 2 — three semantic participants (one in a dual role), real data, a federated catalog node that crawls four providers, and advanced policy enforcement — plus elements of the optional Part 3 in the form of custom ODRL constraints and duties expressed as Java extensions. All claims are backed by reproducible scripts and captured evidence under `final/screenshots/`.

9.2 Future work

- **Production hardening.** TLS / mTLS everywhere, KMS-backed keys, Postgres backups, Helm chart values for prod.
- **More providers.** Wire HKO weather, real MTR (or a synthetic MTR feed if no API is publicly available), and a citizen-reporting feed; this would also stress the federated catalog crawler at a more realistic provider count.
- **Marketplace layer.** Pricing, settlement, and a UI on top of the catalog — the optional Part 3 of the assignment.
- **Consent revocation.** Surface a revocation flow for VCs and verify that connectors honour the revocation when a previously-valid VC is invalidated mid-transfer.
- **Observability.** Prometheus + Grafana dashboards on the connector and dataplane metrics that EDC already exposes.
- **Richer credential vocabularies.** KYC, operating-licence, region-of-incorporation; let policies stack constraints on multiple VCs and expose tools to compose them.

10 Code Repository

The full source code is available at:

<https://github.com/LiChudikang/csc4240-hk-federated-dataspace>

The repository is a fork of `eclipse-edc/MinimumViableDataspace` on main (originally pushed as `csc4240-final`); the upstream remains accessible as the `upstream` remote for traceability. Layout:

```
csc4240-hk-federated-dataspace/      # fork of eclipse-edc/MinimumViableDataspace
|-- extensions/dcp-impl/             # custom Java: ParticipantTierFunction,
|                                     # AttributionDutyFunction,
|                                     # modified PolicyEvaluationExtension
|-- deployment/
|   |-- hku.tf                       # HKU Terraform module (new)
|   |-- assets/credentials/k8s/hku/  # HKU VCs (new)
|-- launchers/dataplane/             # build.gradle.kts: + AWS S3 dataplane
|-- gradle/libs.versions.toml        # + edc-aws version
`-- final/                           # CSC4240 deliverables
    |-- scripts/                     # 8 scripts implementing 7 demos
    |                                 # (Demo 3 uses two)
    |                                 # + minio-deploy.yaml (MinIO manifest, Demo 2)
    |-- screenshots/                 # 24 captured evidence files
    |-- diagrams/                    # scenario / architecture / pipeline (.mmd + .png)
    |-- slides/                      # final-presentation.pptx
    `-- report/                      # this report (report.tex compiled to report.pdf)
```

References

- [1] International Data Spaces Association. *IDSA Reference Architecture Model 4.0*, 2023. <https://docs.internationaldataspaces.org/>.
- [2] Eclipse Foundation. *Eclipse Dataspace Components project page*. <https://projects.eclipse.org/projects/technology.edc>.
- [3] Eclipse Foundation. *Minimum Viable Dataspace*. <https://github.com/eclipse-edc/MinimumViableDataspace>.
- [4] DRM Datenraum Mobilität GmbH. *Mobility Data Space — Germany*. <https://mobility-dataspace.eu/>.
- [5] Catena-X Automotive Network. *Catena-X Operating Model and Technical Whitepaper*, 2023. <https://catena-x.net/>.
- [6] World Wide Web Consortium. *ODRL Information Model 2.2*, W3C Recommendation, 2018. <https://www.w3.org/TR/odrl-model/>.
- [7] World Wide Web Consortium. *Decentralized Identifiers (DIDs) v1.0*, W3C Recommendation, 2022. <https://www.w3.org/TR/did-core/>.
- [8] World Wide Web Consortium. *Verifiable Credentials Data Model v2.0*, W3C Recommendation, 2024. <https://www.w3.org/TR/vc-data-model-2.0/>.
- [9] Kowloon Motor Bus Company. *KMB ETA Open API Specification*, <https://data.etabus.gov.hk/>.

A Demo script index

Table 3: The eight scripts under `final/scripts/` that constitute the reproducible evaluation. `minio-deploy.yaml` (not listed) is the MinIO Kubernetes manifest required by Demo 2.

Demo	Script	Purpose
1	<code>demo-l2-transfer.sh</code>	Negotiation + HTTP-PULL transfer of live KMB ETA
2	<code>demo-push-s3.sh</code>	AmazonS3-PUSH transfer to in-cluster MinIO bucket
3	<code>demo-hku-dual-role.sh</code>	HKU publishes a research asset; HKTaxi pulls it
3	<code>demo-hku-as-consumer.sh</code>	HKU pulls KMB data from <code>provider-qna</code>
4	<code>federated-crawler.sh</code>	Query unified catalog after the Catalog Server crawls
5	<code>demo-live-issuance.sh</code>	DCP issuance of a fresh <code>FoobarCredential</code> to HKU
6	<code>demo-custom-constraint.sh</code>	<code>participantTier</code> permission: positive + negative case
7	<code>demo-attribution-duty.sh</code>	Attribution duty: positive + negative case